1. Phạm vi, tác giả và môi trường

BÁO CÁO PHÂN TÍCH & THIẾT KẾ DEVICE DRIVER — ASSIGNMENT 01

Họ tên: Phạm Võ Hoàng Nhân. MSSV: SE200778. Môn học: LOS201 — Embedded Linux Systems. Bài tập: How to Write a Device Driver, nội dung liên quan Chapter 8–10.

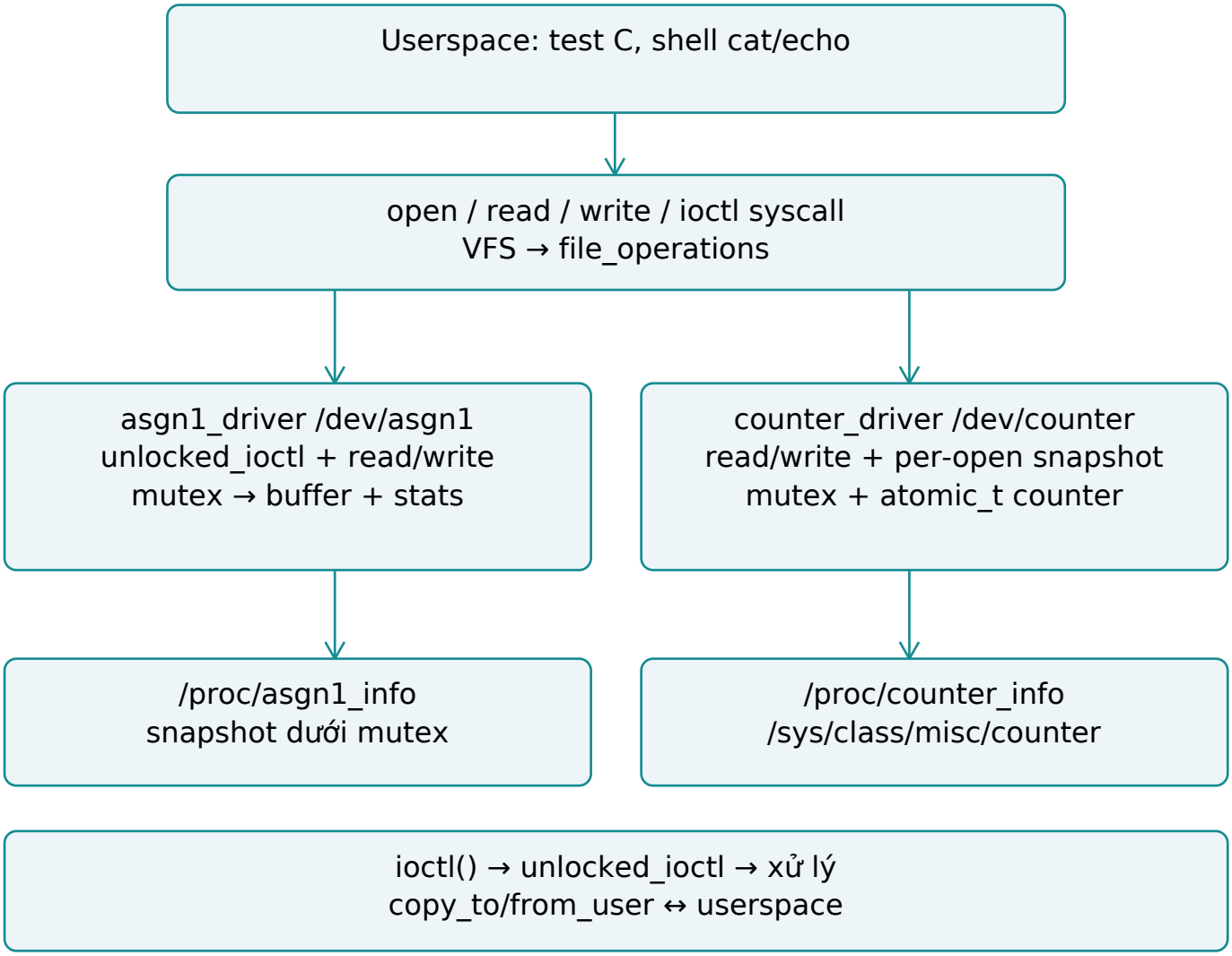
Phạm vi báo cáo gồm thiết kế hai driver, giao diện ioctl và misc device, quy trình build ARM, kiểm thử trên QEMU và phân tích các quyết định kỹ thuật. Mã nguồn có chú thích tiếng Việt để giải thích dữ liệu, tham số, giá trị trả về, khóa đồng bộ và các trường hợp lỗi. Các kết luận chức năng được đối chiếu với log thực tế đi kèm.

Thành phần	Giá trị đã kiểm tra
Host	AlmaLinux 10.2, x86_64, Bash
Guest	QEMU vexpress-a9, Cortex-A9, 512 MB, 2 CPU
Kernel chạy	Linux 5.15.0, dyk@mydklaptop, GCC 15.2.0
Compiler module	Arm GNU Toolchain 13.2.1; arm-none-linux-gnueabihf-
Nguồn build	Linux 5.15.0 nguyên bản; cấu hình kernel dyk đã đối chiếu SHA-256

Môi trường thực hành được tổ chức trong practice/, tách biệt với các file mẫu. Host AlmaLinux dùng để biên dịch chéo; module ARM được nạp và gỡ trong guest QEMU. Cách tổ chức này giúp giữ nguyên tài liệu gốc và truy vết riêng kết quả build, kiểm thử.

Sản phẩm kỹ thuật: hai driver và header ioctl, Makefile, hai executable kiểm thử ARM liên kết tĩnh, script shell, log thật, PDF và file tar.gz. Hồ sơ build và lịch sử lần chạy lỗi cũng được giữ để truy vết.

2. Kiến trúc chung và dòng dữ liệu



Dòng ioctl: chương trình C gọi ioctl(fd, cmd, arg) → syscall → VFS → asgn1_fops.unlocked_ioctl → kiểm tra command → xử lý trạng thái. SET_MODE dùng copy_from_user; GET_STATS và GET_VERSION dùng copy_to_user. Kết quả và mã lỗi quay về userspace.

Shell không tự định nghĩa ABI ioctl; test_asgn1.sh gọi executable asgn1_test dùng chung asgn1_ioctl.h. Counter độc lập, đăng ký bằng misc_register. /sys/class/misc/counter là sysfs do framework tạo; không tuyên bố Task 1 có sysfs attributes riêng.

Data layer gồm buffer/thống kê và atomic counter. Operation layer thực hiện khóa, parse và cập nhật. Interface layer là file_operations, unlocked_ioctl, procfs và chương trình userspace. Sơ đồ mô tả hai driver cùng các giao diện thực sự có trong bản triển khai.

3. Thiết kế Task 1 — ioctl và buffer

Command	Direction / size	Hành vi
RESET_BUFFER	_IO / 0	Xóa buffer; reset read/write count, length và last size; giữ open_count và mode.
GET_STATS	_IOR / 20	Năm int 32-bit; snapshot dưới mutex rồi copy_to_user.
SET_MODE	_IOW / 4	0 thêm [KERNEL]; 1 Echo. Giá trị khác: EINVAL.
GET_VERSION	_IOR / 64	Chuỗi có SE200778; mảng khởi tạo đủ 64 byte, không lộ dữ liệu stack.

Magic là ký tự A (0x41); số command 0–3. Trên ARM EABI dùng ở đây, GET_STATS = 0x80144101, _IOC_NR = 1, _IOC_SIZE = 20; _IOC_DIR = 2 nghĩa là đọc từ kernel về userspace. Size truyền vào macro là kiểu struct, không phải sizeof(struct). Magic giúp phân nhóm, không phải cơ chế bảo mật hay đăng ký độc quyền toàn hệ thống. [1]

Major cố định 241, minor 0; script tạo /dev/asgn1. register_chrdev đăng ký fops, open từ chối minor khác. open_count là số lần mở thành công tích lũy, không phải số descriptor đang mở; read_count chỉ tăng khi thực sự trả dữ liệu. Các bộ đếm bão hòa tại INT_MAX để tránh signed overflow.

Buffer 1024 byte giữ tối đa 1023 byte dữ liệu. Kernel mode dành 8 byte cho [KERNEL], nên nhận tối đa 1015 byte. Ghi quá dài trả EMSGSIZE thay vì cắt âm thầm. memdup_user tạo bản tạm; copy lỗi không phá dữ liệu cũ. memcpy chỉ dùng giữa hai vùng kernel, tuyệt đối không thay copy_to_user trong ioctl.

Tất cả truy cập buffer, stats và mode đều được bảo vệ bằng cùng mutex. RESET không đổi mode/open_count vì đề chỉ yêu cầu reset buffer và read/write count. Mutex được phép ngủ khi xử lý uaccess; không giữ spinlock qua một phép copy có thể page fault. read dùng offset và simple_read_from_buffer để có EOF.

4. Thiết kế Task 2 — misc counter

counter_driver được viết độc lập với Lab 02. struct miscdevice dùng MISC_DYNAMIC_MINOR, major 10 và tên counter. misc_register đơn giản hóa đăng ký thiết bị; trong guest có devtmpfs nên /dev/counter xuất hiện. proc_create bổ sung /proc/counter_info; nếu tạo proc thất bại, init hủy đăng ký misc ngay.

counter, reads và writes dùng atomic_t. Mutex bảo vệ giao dịch lấy giá trị → tạo snapshot → tăng counter, đồng thời phối hợp với thao tác set. atomic_read rồi atomic_inc riêng lẻ không tự tạo thành một giao dịch: hai CPU có thể cùng quan sát giá trị 5 trước khi tăng. Mutex tránh trả trùng snapshot trong trường hợp đó. [3]

Mỗi lần open có read_context riêng. Lần read đầu thành công chụp số, tăng counter một lần; các lần read ngắn tiếp theo trả phần còn lại của cùng chuỗi. Khi offset đạt độ dài, trả 0. Read count=0 hay EFAULT không làm tăng counter. release giải phóng context. Ghi trên descriptor đã đọc không tự tua lại offset; mở lại để đọc snapshot mới.

write nhận ASCII, kiểm tra độ dài và NUL nhúng, dùng kstrtoint(base 10). Số âm, ký tự dư hoặc vượt int trả EINVAL; ghi hợp lệ dùng atomic_set, tăng write_count. write_count tính mọi lần set thành công, kể cả set 100; tên reset trong rubric được hiểu theo đặc tả mục 3.7. Tại INT_MAX, read trả EOVERFLOW để không wrap sang số âm; đây là chính sách biên bổ sung được ghi rõ.

Thời điểm thực tế	counter	reads	writes
Trước write	0	0	0
Sau echo 100	100	2	1
Sau echo 0	0	3	2

Các số trên lấy từ logs/test_counter_output.txt, theo đúng thứ tự script: đọc 0, đọc 1, set 100, đọc 100, reset 0. /proc đọc thống kê không tự tăng counter. Test bổ sung chạy 4 process, mỗi process 20 lần open/read/close: tổng tăng 80, không mất increment.

5. So sánh ba loại driver

Tiêu chí	Character	Block	Network
Giao diện	file_operations cho open/read/write/ioctl	block_device_operations quản lý thiết bị; bio/blk-mq cho I/O	net_device_ops; socket là giao diện ứng dụng
Đơn vị	Byte stream hoặc bản ghi do driver quy định	Block/sector có địa chỉ	Packet/frame
Buffering	Tùy driver; không mặc định có page cache như file thường	Page cache khi buffered I/O, hàng đợi block; direct I/O có thể bỏ qua cache	sk_buff, queue TX/RX, socket buffer, NAPI
Ví dụ nhúng	UART, cảm biến điều khiển, counter	eMMC, SSD, SD có controller	Ethernet MAC, Wi-Fi

VFS không gọi net_device_ops theo cùng cách gọi read của character device. Ứng dụng dùng socket; network stack chuyển packet tới driver. Tương tự, block_device_operations không phải là đường dữ liệu duy nhất của block driver: bio/request và blk-mq xử lý I/O. Bảng giữ các thuật ngữ đề yêu cầu nhưng tránh đồng nhất ba mô hình.

Quyết định 1: thiết bị cảm biến cần đọc vài byte và đổi chế độ đo phù hợp character/misc với read và ioctl; dựng block device sẽ áp đặt sector, địa chỉ và cache không cần thiết. Counter của bài là ví dụ đơn mục đích phù hợp misc.

Quyết định 2: lưu log dạng filesystem trên eMMC dùng block subsystem vì cần truy cập các block và phối hợp filesystem/cache. Nếu thiết bị là Ethernet MAC truyền frame giữa các nút, chọn network subsystem để dùng routing, socket và các cơ chế hàng đợi hiện có.

Task 1 dùng register_chrdev để minh họa major tự quản lý và ABI ioctl. Task 2 dùng misc_register để chia sẻ major 10 và cấp minor động. Misc giảm mã đăng ký/devnode nhưng phù hợp thiết bị nhỏ; character registration riêng linh hoạt hơn khi quản lý nhiều minor hoặc tổ chức lớp thiết bị riêng.

6. Kiểm thử và giới hạn tương thích

Kết quả lấy từ một lần chạy QEMU mới, không nhập lại bằng tay: logs/result.json báo task1=true, task2=true, dyk_kernel=true, completed=true, no_kernel_oops=true và originals_unchanged=true. Việc nạp module không dùng tùy chọn force. Full serial và initramfs của từng lần chạy nằm dưới practice/output/runs/.

Nhóm kiểm tra	Bằng chứng và kết quả
Task 1	Default Echo, Kernel mode, trở lại Echo; 5 trường stats; RESET rồi cat rỗng; version có guard 8 byte; tất cả đạt.
Lỗi và biên Task 1	EINVAL mode sai; ENOTTY command/size sai; EFAULT pointer; EMSGSIZE; 1023 byte và 1015+8 byte; đọc từng phần/EOF; 400 write đồng thời.
Task 2	Đăng ký major 10; 0→1; set 100; reset 0; ba snapshot procfs; read ngắn và EOF; EINVAL/EFAULT; INT_MAX; 80 read đồng thời.
Vòng đời	Cả hai giữ module khi fd mở; đóng fd rồi gỡ; proc/sysfs được dọn; reload hoạt động.
Build/clean	make → clean → make trên bản sao; source còn nguyên, output được tạo lại; log build-clean-verification.txt.

Cấu hình gốc khớp SHA-256 với /proc/config.gz. Compiler module là 13.2.1, compiler kernel dyk là 15.2.0. modules_prepare và vermagic chỉ là kiểm tra sơ bộ. Module.symvers dùng cho liên kết được tạo từ vmlinux build cục bộ, không mạo nhận là file do dyk cung cấp. CONFIG_MODVERSIONS tắt; bằng chứng quan trọng là insmod, chức năng và rmmod thực tế trên kernel dyk đều đạt. [2]

Giới hạn: chưa chứng minh tương thích với mọi kernel 5.15, mọi tải hoặc mọi cấu hình. Không chạy KASAN/kmemleak hay fault injection cấp phát. Tài nguyên được rà soát theo từng nhánh init/exit và kiểm tra reload; không suy diễn “không rò rỉ trong mọi trường hợp”. Cảnh báo nền phần cứng QEMU vẫn được giữ trong dmesg; không có kernel Oops/panic trong lượt kiểm thử.

7. Lỗi thực tế và cách sửa

Lỗi 1 — build thất bại vì trùng asgn1_open. DEFINE_PROC_SHOW_ATTRIBUTE(asgn1) tự sinh một hàm asgn1_open cho procfs, trùng callback open của character device. Log gốc: logs/task1-build.txt. Sửa bằng namespace asgn1_status_show / DEFINE_PROC_SHOW_ATTRIBUTE(asgn1_status); áp dụng cùng nguyên tắc cho counter. Build lại thành công.

```
error: redefinition of ‘asgn1_open’
note: in definition of macro ‘DEFINE_PROC_SHOW_ATTRIBUTE’
```

Lỗi 2 — cảnh báo stack frame 1032 byte vượt ngưỡng 1024. Bản đầu đặt input[1024] trên kernel stack. Log: logs/task1-build-fixed.txt. Chuyển sang memdup_user và kfree ở cả đường thành công lẫn đường EMSGSIZE. Không giữ buffer tạm trên stack lớn; uaccess lỗi trả errno trước khi cập nhật trạng thái.

Lỗi 3 — script báo thất bại dù kernel từ chối unload đúng. Trong lượt 20261017-021607, BusyBox rmmod in “Resource temporarily unavailable” nhưng nhánh if coi exit status là thành công. Hai chương trình C đã đạt, nhưng shell test kết luận sai. Không sửa driver để vượt kiểm tra refcount; sửa script để kiểm tra trạng thái có thật trong /proc/modules.

```
rmmod asgn1_driver || true
grep '^asgn1_driver ' /proc/modules
exec 9>&-
rmmod asgn1_driver
if grep -q '^asgn1_driver ' /proc/modules; then exit 1; fi
```

Sau sửa, cả hai script có marker PASS, proc entry biến mất, counter sysfs/devnode biến mất và dmesg báo cleanup complete. Lượt lỗi vẫn được giữ bên cạnh lượt thành công; không xóa hay sửa log để che kết quả ban đầu.

Khác biệt cấu hình cũng đã được xử lý: GCC 13.2 phát hiện plugin headers nên olddefconfig tự bật GCC_PLUGINS và STACKPROTECTOR_PER_TASK. Tắt GCC_PLUGINS trong bản thực hành đưa các tùy chọn chức năng về trạng thái của dyk; diff còn thông tin toolchain và dòng tắt plugin.

8. Lý thuyết — reference count và uaccess

Câu 1. Module có reference count để mã và dữ liệu không bị giải phóng khi còn người dùng. Với .owner = THIS_MODULE, lấy file_operations qua fops_get gọi try_module_get; giải phóng qua fops_put gọi module_put. Lần open tạo file object giữ tham chiếu; close cuối cùng mới giải phóng. dup/fork chia sẻ file object, không nhất thiết tăng module count đúng một lần cho mỗi descriptor. Phụ thuộc giữa module và tham chiếu tường minh cũng có thể giữ module.

rmmod khi còn fd mở thường bị từ chối vì module còn được dùng; trong test kernel giữ module, BusyBox in Resource temporarily unavailable. Đóng fd rồi gỡ thành công. Về lý thuyết có rmmod -f nếu kernel bật CONFIG_MODULE_FORCE_UNLOAD và tiến trình có quyền cần thiết; cưỡng bức có thể để lại con trỏ vào mã đã giải phóng. Kernel dyk của bài tắt tùy chọn này, nên không thể dựa vào force unload. Không thực hiện force unload trong kiểm thử.

Câu 2. Con trỏ userspace là địa chỉ ảo không đáng tin cậy, phụ thuộc address space hiện hành. MMU dịch địa chỉ qua page table và kiểm tra quyền; trang có thể chưa được ánh xạ, chỉ đọc, bị thay đổi hoặc cần xử lý page fault. memcpy thông thường không cung cấp hợp đồng uaccess và cơ chế sửa lỗi truy cập phù hợp để trả lỗi cho syscall.

Không đúng khi khẳng định kernel “luôn không thể” truy cập địa chỉ user hoặc “luôn panic”: kết quả tùy mapping và cơ chế bảo vệ của kiến trúc. Truy cập trực tiếp có thể tình cờ hoạt động; cũng có thể gây translation/permission fault không được phục hồi đúng, dẫn tới Oops, hoặc ghi nhầm vùng nếu bỏ kiểm tra địa chỉ. copy_to_user dùng kiểm tra và đường exception fixup của kiến trúc, trả số byte chưa copy; driver đổi kết quả khác 0 thành -EFAULT.

GET_STATS chụp struct dưới mutex rồi copy_to_user snapshot. Không trả con trỏ kernel cho userspace. Struct gồm 5 int, không có trường pointer hay dữ liệu padding chưa khởi tạo trong ABI ARM này. GET_VERSION khởi tạo cả mảng 64 byte trước khi copy. Test pointer 0x1 cho GET_STATS, SET_MODE và GET_VERSION trả EFAULT mà kernel vẫn chạy.

Nguồn đối chiếu đúng Linux 5.15: include/linux/fs.h (fops_get/fops_put), kernel/module.c (try_stop_module, try_force_unload), include/linux/uaccess.h và arch/arm/include/asm/uaccess.h. Đây là giải thích kỹ thuật; không phải tuyên bố đã mô phỏng mọi lỗi MMU.

9. Lý thuyết — MTD, offset và tài liệu

Câu 3. MTD biểu diễn flash thô với erase block, yêu cầu erase trước program, giới hạn chu kỳ và các đặc điểm như bad block/OOB/ECC ở NAND. Block subsystem cung cấp địa chỉ sector và I/O block; thiết bị như eMMC/SSD có controller/FTL che việc quản lý flash nên thường đi qua block layer. Không phải cứ lưu trữ trên flash là phải dùng MTD.

Ví dụ: chip SPI NOR chứa firmware hoặc raw NAND nối trực tiếp SoC cần MTD và driver/controller thích hợp. Raw NAND có thể dùng UBI/UBIFS để quản lý phía trên MTD. Thẻ SD/eMMC trình bày block interface thì dùng block layer. mtdblock là lớp chuyển tiếp, không tự biến raw flash thành ổ đĩa có đầy đủ FTL, wear leveling và độ tin cậy tương đương SSD. Task 4 chỉ yêu cầu giải thích; không chạy flash_erase/nandsim trên host.

Câu 4. cat lặp syscall read cho tới khi nhận 0 (EOF) hoặc lỗi. Mỗi syscall đi qua VFS rồi callback read của driver; VFS không tự lặp vô hạn bên trong một lần gọi để thay cat. Nếu callback luôn trả chuỗi số với số byte dương, cat sẽ tiếp tục gọi và in dữ liệu mãi, có thể làm counter tăng liên tục.

counter_read dùng snapshot riêng cho mỗi open và *ppos. Lần đọc đầu trả một phần hoặc toàn bộ chuỗi, tăng counter đúng một lần sau khi copy thành công. Các lần đọc sau trả phần còn lại; khi offset bằng độ dài thì trả 0. Điều này xử lý cả cat lẫn ứng dụng đọc từng byte. Test đã xác nhận “100\n” được đọc theo 1+3 byte, lần kế tiếp EOF và counter chỉ tăng lên 101.

Tài liệu sử dụng: đề ASSIGNMENT01_EmbeddedLinux.docx, mục 3.2–3.10 và IV–V; README/BUILD_FROM_SCRATCH của Lab 01/02 trong repository. Thiết kế mới có tham khảo khái niệm Lab 02, không sao chép code từ GitHub/Stack Overflow ngoài repository được cung cấp.

[1] Linux Kernel Documentation, ioctl based interfaces: https://docs.kernel.org/driver-api/ioctl.html. [2] Building External Modules: https://docs.kernel.org/kbuild/modules.html. [3] Atomic types: https://docs.kernel.org/core-api/wrappers/atomic_t.html. Bản tài liệu đi kèm Linux 5.15 được ưu tiên khi API khác phiên bản hiện hành.

Các nguồn mã bổ sung: drivers/char/misc.c; include/linux/atomic/; fs/read_write.c; fs/libfs.c; include/linux/mtd/mtd.h và Documentation/driver-api/mtd/. Các đường dẫn này nằm trong cây mã nguồn Linux 5.15 đã dùng để chuẩn bị môi trường build.

Phụ lục A — toàn bộ log Task 1

Nguồn: logs/test_asgn1_output.txt. Đoạn 1; chỉ bỏ mã màu và xuống dòng để vừa trang.

```
+ insmod ./asgn1_driver.ko
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: loaded major=241 mode=1
+ grep '^asgn1_driver ' /proc/modules
asgn1_driver 16384 0 - Live 0x7f000000 (O)
+ '[' '!' -c /dev/asgn1 ]
+ mknod /dev/asgn1 c 241 0
+ ls -l /dev/asgn1
crw-r--r--    1 root     0         241,   0 Oct 17 06:29 /dev/asgn1
+ ./asgn1_test
PASS fd >= 0
ABI stats_size=20 GET_STATS=0x80144101 nr=1 size=20
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=1 write_count=0 read_count=0 buffer_len=0
last_write_size=0
PASS s.open_count == 1 && s.write_count == 0 && s.read_count == 0 &&
s.buffer_len == 0
PASS write(fd,"hello",5) == 5
PASS fd >= 0
PASS n == (ssize_t)strlen(expected)
PASS !strcmp(text, expected)
PASS read(fd, text, sizeof(text)) == 0
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=2 write_count=1 read_count=1 buffer_len=5
last_write_size=5
PASS s.write_count == 1 && s.read_count == 1 && s.buffer_len == 5 &&
s.last_write_size == 5
asgn1_driver: ioctl cmd=0x40044102 nr=2
PASS ioctl(fd,ASGN1_SET_MODE,&mode) == 0
PASS write(fd,"abc",3) == 3
PASS fd >= 0
PASS n == (ssize_t)strlen(expected)
PASS !strcmp(text, expected)
PASS read(fd, text, sizeof(text)) == 0
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=3 write_count=2 read_count=2 buffer_len=11
last_write_size=3
PASS s.buffer_len == 11 && s.last_write_size == 3
asgn1_driver: ioctl cmd=0x40044102 nr=2
PASS ioctl(fd,ASGN1_SET_MODE,&mode) == 0
PASS write(fd,"echo",4) == 4
```

Phụ lục A — toàn bộ log Task 1

Nguồn: logs/test_asgn1_output.txt. Đoạn 2; chỉ bỏ mã màu và xuống dòng để vừa trang.

```
PASS fd >= 0
PASS n == (ssize_t)strlen(expected)
PASS !strcmp(text, expected)
PASS read(fd, text, sizeof(text)) == 0
asgn1_driver: ioctl cmd=0x80404103 nr=3
PASS ioctl(fd,ASGN1_GET_VERSION,version.text) == 0
PASS memchr(version.text,0,64) != NULL
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
PASS version.guard[i] == 0xa5
VERSION asgn1_driver v1.0 - SE200778
asgn1_driver: ioctl cmd=0x40044102 nr=2
PASS ioctl(fd,ASGN1_SET_MODE,&mode) == -1 && errno == EINVAL
asgn1_driver: ioctl cmd=0x4163 nr=99
PASS ioctl(fd,_IO(ASGN1_MAGIC,99),0) == -1 && errno == ENOTTY
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd,ASGN1_GET_STATS,(void *)1) == -1 && errno == EFAULT
asgn1_driver: ioctl cmd=0x40044102 nr=2
PASS ioctl(fd,ASGN1_SET_MODE,(void *)1) == -1 && errno == EFAULT
asgn1_driver: ioctl cmd=0x80404103 nr=3
PASS ioctl(fd,ASGN1_GET_VERSION,(void *)1) == -1 && errno == EFAULT
asgn1_driver: ioctl cmd=0x80014101 nr=1
PASS ioctl(fd,_IOR(ASGN1_MAGIC,1,char),&s) == -1 && errno == ENOTTY
PASS write(fd,large,sizeof(large)) == -1 && errno == EMSGSIZE
PASS fd >= 0
PASS n == (ssize_t)strlen(expected)
PASS !strcmp(text, expected)
PASS read(fd, text, sizeof(text)) == 0
PASS syscall(SYS_write,fd,(void *)1,4) == -1 && errno == EFAULT
PASS fd >= 0
PASS n == (ssize_t)strlen(expected)
PASS !strcmp(text, expected)
PASS read(fd, text, sizeof(text)) == 0
PASS write(fd,large,sizeof(large)-1) == sizeof(large)-1
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=6 write_count=4 read_count=5 buffer_len=1023
last_write_size=1023
PASS s.buffer_len == 1023
```

Phụ lục A — toàn bộ log Task 1

Nguồn: logs/test_asgn1_output.txt. Đoạn 3; chỉ bỏ mã màu và xuống dòng để vừa trang.

```
asgn1_driver: ioctl cmd=0x40044102 nr=2
PASS ioctl(fd,ASGN1_SET_MODE,&mode) == 0
PASS write(fd,large,1016) == -1 && errno == EMSGSIZE
PASS write(fd,large,1015) == 1015
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=6 write_count=5 read_count=5 buffer_len=1023
last_write_size=1015
PASS s.buffer_len == 1023 && s.last_write_size == 1015
asgn1_driver: ioctl cmd=0x4100 nr=0
PASS ioctl(fd,ASGN1_RESET_BUFFER,0) == 0
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=6 write_count=0 read_count=0 buffer_len=0
last_write_size=0
PASS s.write_count == 0 && s.read_count == 0 && s.buffer_len == 0 &&
s.last_write_size == 0
PASS s.open_count > 0
PASS fd >= 0
PASS n == (ssize_t)strlen(expected)
PASS !strcmp(text, expected)
PASS read(fd, text, sizeof(text)) == 0
asgn1_driver: ioctl cmd=0x40044102 nr=2
PASS ioctl(fd,ASGN1_SET_MODE,&mode) == 0
PASS write(fd,"abcdef",6) == 6
PASS fd >= 0
PASS read(rd,small,2) == 2
PASS read(rd,small+2,4) == 4
PASS !strcmp(small,"abcdef")
PASS read(rd,small,1) == 0
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=8 write_count=1 read_count=2 buffer_len=6
last_write_size=6
PASS children[i] >= 0
PASS children[i] >= 0
PASS children[i] >= 0
PASS children[i] >= 0
PASS children[i] >= 0
PASS children[i] >= 0
PASS children[i] >= 0
PASS waitpid(children[i],&status,0) == children[i]
PASS WIFEXITED(status) && WEXITSTATUS(status) == 0
PASS waitpid(children[i],&status,0) == children[i]
```

Phụ lục A — toàn bộ log Task 1

Nguồn: logs/test_asgn1_output.txt. Đoạn 4; chỉ bỏ mã màu và xuống dòng để vừa trang.

```
PASS WIFEXITED(status) && WEXITSTATUS(status) == 0
PASS children[i] >= 0
PASS waitpid(children[i],&status,0) == children[i]
PASS WIFEXITED(status) && WEXITSTATUS(status) == 0
PASS waitpid(children[i],&status,0) == children[i]
PASS WIFEXITED(status) && WEXITSTATUS(status) == 0
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=12 write_count=401 read_count=2 buffer_len=4
last_write_size=4
PASS s.write_count == before.write_count+400
PASS s.open_count == before.open_count+4
asgn1_driver: ioctl cmd=0x4100 nr=0
PASS ioctl(fd,ASGN1_RESET_BUFFER,0) == 0
asgn1_driver: ioctl cmd=0x80144101 nr=1
PASS ioctl(fd, ASGN1_GET_STATS, &s) == 0
STATS open_count=12 write_count=0 read_count=0 buffer_len=0
last_write_size=0
PASS s.buffer_len == 0 && s.write_count == 0 && s.read_count == 0
TASK1_C_TESTS_PASS
+ cat /dev/asgn1
+ test -z
+ exec
+ rmmod asgn1_driver
rmmod: remove 'asgn1_driver': Resource temporarily unavailable
+ grep '^asgn1_driver ' /proc/modules
asgn1_driver 16384 2 - Live 0x7f000000 (O)
+ exec
+ rmmod asgn1_driver
asgn1_driver: cleanup complete
input: ImExPS/2 Generic Explorer Mouse as
/devices/platform/bus@40000000/bus@40000000:motherboard-
bus@40000000/bus@40000000:motherboard-
bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
+ test '!' -e /proc/asgn1_info
+ grep -q '^asgn1_driver ' /proc/modules
+ + grep asgn1_driver:
dmesg
asgn1_driver: loading out-of-tree module taints kernel.
asgn1_driver: loaded major=241 mode=1
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x40044102 nr=2
asgn1_driver: ioctl cmd=0x80144101 nr=1
```

Phụ lục A — toàn bộ log Task 1

Nguồn: logs/test_asgn1_output.txt. Đoạn 5; chỉ bỏ mã màu và xuống dòng để vừa trang.

```
asgn1_driver: ioctl cmd=0x40044102 nr=2
asgn1_driver: ioctl cmd=0x80404103 nr=3
asgn1_driver: ioctl cmd=0x40044102 nr=2
asgn1_driver: ioctl cmd=0x4163 nr=99
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x40044102 nr=2
asgn1_driver: ioctl cmd=0x80404103 nr=3
asgn1_driver: ioctl cmd=0x80014101 nr=1
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x40044102 nr=2
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x4100 nr=0
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x40044102 nr=2
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: ioctl cmd=0x4100 nr=0
asgn1_driver: ioctl cmd=0x80144101 nr=1
asgn1_driver: cleanup complete
+ dmesg
+ grep 'asgn1_driver: cleanup complete'
asgn1_driver: cleanup complete
+ insmod ./asgn1_driver.ko
asgn1_driver: loaded major=241 mode=1
+ cat /dev/asgn1
+ test -z
+ printf default-echo
asgn1_driver: ioctl cmd=0x802c542a nr=42
+ cat /dev/asgn1
+ test default-echo '=' default-echo
+ rmmod asgn1_driver
asgn1_driver: cleanup complete
+ echo TASK1_SCRIPT_PASS
TASK1_SCRIPT_PASS
ASM1_TASK1_RC=0
```

Phụ lục B — minh chứng và gói bàn giao

Các file sau chứa dữ liệu nguyên bản, không phải output giả định: logs/test_asgn1_output.txt, logs/test_counter_output.txt, logs/dmesg_full.txt, logs/runtime.txt, logs/result.json và logs/module-metadata.txt. Log Task 1 đầy đủ ở phụ lục A; log Task 2 đầy đủ và toàn bộ dmesg đi kèm archive.

Trích dmesg thực tế sau kiểm thử và cleanup:

```
asgn1_driver: cleanup complete
asgn1_driver: cleanup complete
counter_driver: cleanup complete
counter_driver: cleanup complete
```

Kết quả kiểm tra tự động:

```
{
  "qemu_returncode": 0,
  "task1": true,
  "task2": true,
  "dyk_kernel": true,
  "no_kernel_oops": true,
  "originals_unchanged": true,
  "completed": true
}
```

Tên archive: SE200778_PhamVoHoangNhan_Asgn01.tar.gz. PDF riêng nộp theo mục 4.4: SE200778_PhamVoHoangNhan_Asgn01_BaoCao.pdf. Trong bao_cao/ giữ SE200778_Asgn01_BaoCao.pdf theo cây thư mục mục 4.1; hai bản có cùng nội dung.

Đề không quy định số ảnh chụp riêng. Minh chứng gồm log kiểm thử thực tế, metadata module và sơ đồ thiết kế. Deadline cần đối chiếu trên LMS; bộ tài liệu này chưa được tự động nộp lên hệ thống của trường.